

School Management System

Technical Documentation & Developer Handover Guide
Cameroon Anglophone Secondary (GCE) · PHP 8 Custom MVC

Repository: [tebitramson69-netizen/School-Management-System](https://github.com/tebitramson69-netizen/School-Management-System)

Author: Tebit Ramson Titih

Stack: PHP 8 · MySQL/MariaDB (PDO) · Bootstrap 5.3 · Vanilla JS · Apache (XAMPP)

Size: 74 files · ~19,300 lines · 4 user roles · 12 tables

Documented at commit: [e9f23ca](https://github.com/tebitramson69-netizen/School-Management-System/commit/e9f23ca)

Document version: 1.0 · 21 September 2026

Audience: Developers joining, reviewing or deploying this codebase

Table of Contents

- 1. Executive Summary**
- 2. Domain Primer — The Cameroon GCE System**
 - Why a developer must read this first
 - Terms, sequences and the 0-20 scale
 - Coefficients and weighted averages
- 3. Technology Stack & Runtime**
- 4. Project Structure**
- 5. Architecture**
 - Layering
 - Front controller & request lifecycle
 - The view layout system
- 6. Routing Reference**
- 7. Authentication & Authorization**
 - Login flow
 - Role enforcement
 - Forced password change
 - CSRF, rate limiting, session hardening
- 8. Database Design**
 - Entity-relationship overview
 - Table reference
 - The terms/sequence modelling decision
 - Migrations
- 9. The Academic Engine**
 - Score entry
 - Subject averages and term aggregation
 - Coefficient-weighted overall average
 - Grade scale
 - Class rankings
- 10. Feature Walkthrough by Role**
- 11. Front-End Architecture**
- 12. Installation & Setup**
- 13. Security Review**
- 14. Verified Defects & Root-Cause Analysis**
- 15. Documentation Drift**
- 16. Improvement Roadmap**
- 17. Appendix A — Route Table**
- 18. Appendix B — Model API Reference**
- 19. Appendix C — Glossary**

1. Executive Summary

This is a custom-built, framework-free PHP 8 MVC application that manages the academic operations of a Cameroonian Anglophone secondary school: user accounts across four roles, classes and subjects, teacher assignments, attendance, score entry per sequence, coefficient-weighted averages, grading, class rankings, report cards and announcements.

It is a genuinely substantial system — roughly 19,300 lines across 74 files, with a clean separation between a front controller, controllers, models, middleware and views. It is not a tutorial project. The domain logic in particular (term/sequence aggregation, coefficient weighting, competition-style ranking) reflects real understanding of how Cameroonian secondary schools actually compute results.

Current state at a glance

Area	State	Summary
Authentication	Solid	Session-based, hashed passwords, session regeneration on login, rate limiting, forced password change on first login.
Authorization	Solid	Role middleware on every controller action; ownership checks on teacher assignments and parent-child links.
CSRF protection	Solid	Validated globally for every POST in the front controller — one chokepoint, impossible to forget per-form.
SQL injection defence	Solid	PDO prepared statements with named parameters throughout. No string-concatenated SQL found.
Output escaping	Solid	167 htmlspecialchars() calls with ENT_QUOTES ; numeric output int-cast before echo.
Academic engine	Mostly right	Weighted averages and rankings are implemented and share one definition. Pass/fail threshold is hard-coded and contradicts the configurable setting (§14.2).
Deployability	Blocked	A fresh clone cannot boot: config/env.php is required but git-ignored (§14.1). One include also breaks on Linux due to file casing (§14.3).
Web-root exposure	Critical	.htaccess is empty; with the documented BASE_URL, .env and all source are fetchable over HTTP (§13.1).
Administrative CRUD	Partial	Create works for all entities; no edit, deactivate or delete anywhere.
Automated tests	None	No test suite, no CI workflow.

The honest one-paragraph verdict. The security fundamentals and the domain modelling are well above what this kind of project usually shows — CSRF at a single chokepoint, prepared statements without exception, and a ranking engine deliberately built to share one averaging definition with the report card are all senior-level decisions. What stands between this and a deployable product is not architecture; it is three concrete blockers (§13.1, §14.1, §14.3), a hard-coded pass mark that is wrong for Cameroon (§14.2), and the absence of edit/delete operations. Those are days of work, not months.

2. Domain Primer — The Cameroon GCE System

2.1 Why a developer must read this first

Most of the non-obvious code in this project exists because of how Cameroonian schools grade. A developer who skips this section will read the `terms` table or `Score::subjectAveragesForStudentTerm()` and conclude they are buggy. They are not — they encode real rules.

2.2 Terms, sequences and the 0-20 scale

Concept	Meaning in this system
Cycles	First Cycle covers Forms 1-5, ending with the GCE Ordinary Level at Form 5. Second Cycle covers Lower Sixth and Upper Sixth, ending with the GCE Advanced Level.
Academic year	Named like <code>2026/2027</code> . Exactly one is flagged current at a time.
Term	Three per academic year: Term 1, Term 2, Term 3.
Sequence	Each term contains two assessment sequences, giving six sequences per year. A subject's term mark is the mean of its two sequence marks.
Marking scale	Every mark is out of 20. The database stores <code>score</code> and <code>max_score</code> , with <code>max_score</code> fixed at <code>20.00</code> on insert.
Pass mark	10 out of 20 is the standard threshold. It is configurable in <code>school_settings.pass_mark</code> — but see §14.2, where the code ignores that column.

2.3 Coefficients and weighted averages

Subjects are not equal. A science stream weights Physics and Mathematics more heavily than, say, Food Science. Each subject carries a *coefficient* — an integer multiplier — and the overall average is:

$$\text{overall average} = \Sigma(\text{subject_average} \times \text{coefficient}) / \Sigma(\text{coefficients})$$

Coefficients are standardised by MINESEC but vary by class and stream, so this system stores them per subject *per class* in the `subject_coefficients` table and lets an administrator edit them. A plain arithmetic mean of subject marks is **mathematically wrong** for a Cameroonian report card — which is why the weighting logic in §9.3 matters so much.

3. Technology Stack & Runtime

Layer	Technology	Notes
Language	PHP 8+	Typed properties, union return types (<code>array false</code>), <code>declare(strict_types=1)</code> in newer files. No Composer, no autoloader — explicit <code>require_once</code> everywhere.
Database	MySQL / MariaDB via PDO	Singleton connection (<code>Database::getConnection()</code>), <code>utf8mb4</code> , exceptions on error, default fetch mode <code>FETCH_ASSOC</code> .
Web server	Apache (XAMPP)	Document root is the project's parent; the app is served from <code>public/</code> via <code>BASE_URL</code> .
CSS framework	Bootstrap 5.3.3 (CDN)	Supplemented by 5 local stylesheets totalling ~4,900 lines.
JavaScript	Vanilla, one file	<code>public/js/app.js</code> (944 lines) — no framework, no build step, no bundler.
Configuration	<code>.env</code> + <code>config/env.php</code>	Credentials externalised from code. See §14.1 for the blocker this currently creates.
Testing / CI	None	No PHPUnit, no GitHub Actions workflow.

Verified: all 74 PHP files pass `php -l` syntax checking under PHP 8.4.

4. Project Structure

School-Management-System/	
├── .env.example	Template for local credentials
├── .gitignore	Ignores .env AND config/env.php ← see §14.1
├── .htaccess	EMPTY (0 bytes) ← see §13.1
├── CLAUDE.md	Architecture + conventions notes
├── PROJECT_PROGRESS.md	Phase log and roadmap
├── config/	
│ ├── database.php	PDO singleton + BASE_URL/SCHOOL_EMAIL_DOMAIN constants
│ └── env.php	.env parser – NOT IN THE REPOSITORY (git-ignored)
├── database/	
│ ├── schema.sql	Full schema, 12 tables + seed data
│ ├── migration_001_sync_schema.sql	Adds columns the app expected
│ ├── migration_002_pass_mark.sql	Adds school_settings.pass_mark
│ └── migration_003_terms_unique_constraint.sql	
├── public/	← the only directory that should be web-reachable
│ ├── index.php	Front controller / router (707 lines, 30 routes)
│ ├── css/ theme · components	· dashboard · auth · responsive (~4,900 lines)
│ └── js/app.js	Alerts, sidebar, dirty-form guard, print, badges
├── src/	
│ ├── Core/	
│ │ ├── Security.php	CSRF, session hardening, password policy, rate limiting
│ │ └── School.php	Cached accessor for school_settings
│ ├── Middleware/	
│ │ └── AuthMiddleware.php	requireRole(), 403 page, password-change redirect
│ ├── Controllers/	Auth · Admin · Teacher · Student · Parent · AcademicYear
│ └── Models/	16 models, one per table/aggregate
├── views/	
│ ├── layouts/dashboard.php	Shared chrome used by every page
│ ├── components/	header.php · sidebar.php · stat-card.php
│ ├── auth/ login · change_password	
│ ├── admin/ 13 views	
│ ├── teacher/ dashboard · Mark_attendance · enter_scores	
│ ├── student/ information · report_card · attendance · announcements · dashboard	
│ └── parent/ dashboard · child_detail	
└── images/	Logo and background

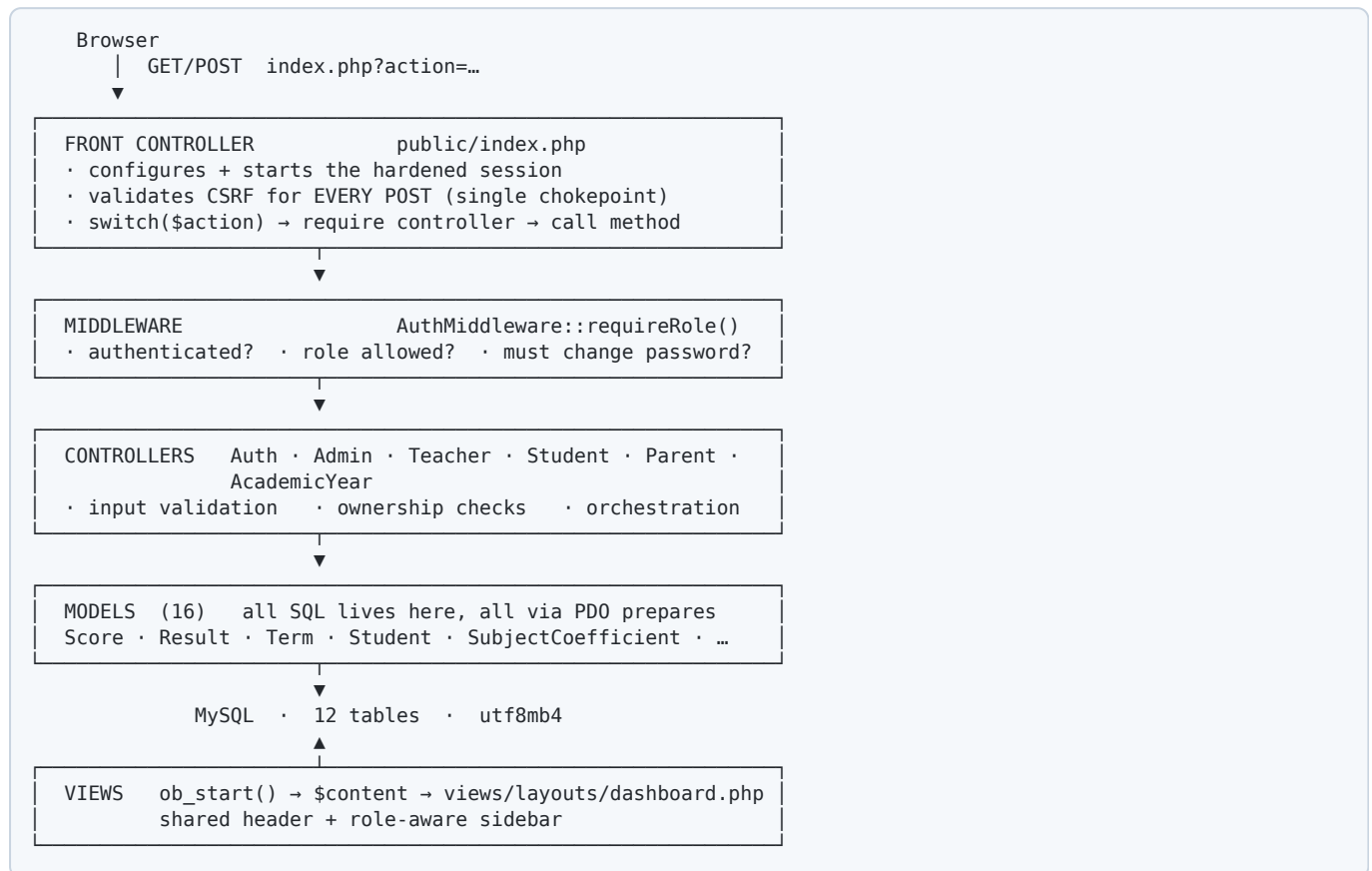
Size distribution

Component	Lines	Largest files
CSS	~4,940	auth.css 1,930 · components.css 1,323 · dashboard.css 931
Controllers	~3,250	AdminController 1,282 · StudentController 830 · AcademicYearController 449
Models	~2,900	Score 761 · Result 462 · Term 423 · Student 368
Views	~3,900	sidebar.php 1,195 · admin/dashboard.php 589 · header.php 559
JavaScript	944	app.js
Front controller	707	public/index.php

5. Architecture

5.1 Layering

The application is a hand-rolled MVC with four clear layers and no framework underneath:



The layering is respected. Every SQL statement found in the codebase lives in `src/Models/`. Controllers never query the database directly, and views never do either — they receive prepared variables. That discipline is what makes this codebase extensible, and it should be defended in review.

5.2 Front controller & request lifecycle

Every request enters through `public/index.php` and follows the same path:

1. **Configuration** — `config/database.php` loads `.env` and defines `BASE_URL` and `SCHOOL_EMAIL_DOMAIN`.
2. **Session hardening** — `Security::configureSession()` sets `httponly`, `samesite=Strict`, `use_strict_mode` and `use_only_cookies` *before* `session_start()`. Order matters: these `ini_set()` calls are no-ops once a session exists, which is why the method returns early if one is already active.
3. **Action resolution** — `$action = trim($_GET['action'] ?? 'login')`, defaulting to the login page.
4. **Global CSRF gate** — if the method is POST, `Security::requireValidCsrf()` runs before any route is dispatched.
5. **Dispatch** — a `switch` maps the action to a controller method. Controllers are `require d` inside their own case, so a login request never loads the admin controller or its models.

6. **Authorization** — the controller method's first statement is `AuthMiddleware::requireRole(...)`.
7. **Response** — write actions redirect (Post/Redirect/Get); read actions render a view through the shared layout.

Global CSRF validation is the single best decision in this codebase. Most PHP projects validate CSRF inside each handler, and the first handler someone forgets becomes the vulnerability. Here it is structurally impossible to add an unprotected POST route — the gate sits above the router. Preserve this property in any refactor.

5.3 The view layout system

Views use output buffering to inject their body into shared chrome:

```
$pageTitle = 'Admin Dashboard';
ob_start();
// ... the page's own HTML ...
$content = ob_get_clean();
require __DIR__ . '/../layouts/dashboard.php';
```

`views/layouts/dashboard.php` then renders `components/header.php`, the role-aware `components/sidebar.php`, and drops `$content` into the main region. Verified: **every one of the 23 page views uses this layout**. The only file that does not is `views/student/_header.php`, a partial, whose leading underscore correctly signals that.

Note: both `CLAUDE.md` and `PROJECT_PROGRESS.md` still describe layout migration as outstanding work. It is complete — see §15.

6. Routing Reference

Routing is query-string based: `public/index.php?action=<name>`. There is no URL rewriting. Thirty actions are registered; an unknown action returns a styled HTTP 404.

Action	Role	Handler	Purpose
Authentication			
login	public	AuthController::showLoginForm	Login page; redirects if already signed in.
login_submit	public	AuthController::login	Authenticate; rate-limited.
logout	any	AuthController::logout	Clear session + cookie, destroy, redirect.
change_password_form	any	AuthController::showChangePasswordForm	Password change page.
change_password	any	AuthController::changePassword	Validate strength and update.
Admin			
admin_dashboard	admin	AdminController::dashboard	Counts and overview.
academic_years	admin	AcademicYearController::index	Years, terms, sequences.
create_academic_year	admin	AcademicYearController::create	Create a year (generates terms).
activate_academic_year	admin	AcademicYearController::activate	Make a year current.
set_current_term	admin	AcademicYearController::setCurrentTerm	Set current term/sequence.
create_teacher_form / create_teacher	admin	AdminController	Teacher account + profile.
create_student_form / create_student	admin	AdminController	Student account, profile and enrolment (transactional).
create_parent_form / create_parent	admin	AdminController	Parent account + child links.
assign_teacher_form / assign_teacher	admin	AdminController	Bind teacher↔subject↔class.
class_coefficients_form / save_class_coefficients	admin	AdminController	Per-class subject coefficients.
view_teachers / view_classes / view_class_list	admin	AdminController	Read-only listings.
post_announcement_form / post_announcement / view_announcements	admin	AdminController	School-wide or per-class notices.
Teacher			
teacher_dashboard	teacher	TeacherController::dashboard	Assignments + announcements.
mark_attendance_form / mark_attendance	teacher	TeacherController	Daily register per assignment.

enter_scores_form / enter_scores	teacher	TeacherController	Bulk marks for a term+sequence.
Student			
student_information	student	StudentController::information	Profile and class.
student_report_card	student	StudentController::reportCard	Subject averages, overall, position.
student_attendance	student	StudentController::attendance	Attendance history.
student_announcements	student	StudentController::announcements	School + class notices.
student_dashboard	student	—	HTTP 301 to student_information (legacy bookmarks).
Parent			
parent_dashboard	parent	ParentController::dashboard	List of linked children.
parent_child_detail	parent	ParentController::childDetail	One child's results, attendance, position.

Scaling note. The router is a 707-line `switch` in which each case repeats five lines of boilerplate. It works and it is readable, but it grows linearly with every feature. A route table (`'action' => [Controller::class, 'method', 'role']`) driven by a small dispatch loop would collapse those 707 lines to roughly 40 and make the role requirement visible in one place. Worth doing before Phase 2 adds another thirty routes — not urgent before that.

7. Authentication & Authorization

7.1 Login flow

`AuthController::login()` executes these checks in order, and each failure exits immediately:

1. Reject non-POST requests.
2. `Security::checkLoginRateLimit()` — blocks after 5 failures for 5 minutes.
3. Both fields present; email passes `FILTER_VALIDATE_EMAIL`.
4. Look up the user; verify with `password_verify()`.
5. On failure, record the attempt and return the *same* message — `"Invalid email or password."` — whether the email exists or not. This is deliberate: a distinct "no such account" message would let an attacker enumerate valid addresses.
6. Reject deactivated accounts (`is_active = 0`).
7. `session_regenerate_id(true)` — issues a new session ID on privilege change, defeating session fixation.
8. Store only `user_id`, `email`, `role`, `must_change_password` in the session. No password hash, no full user row.
9. Redirect to the role's dashboard; an unrecognised role triggers a logout rather than falling through.

7.2 Role enforcement

`AuthMiddleware::requireRole(string|array $allowedRoles)` is the single authorization primitive. It verifies a session exists, normalises the allowed roles to an array, and compares with `in_array($currentRole, $allowedRoles, true)` — strict comparison, so no type-juggling bypass. Unauthorised users get HTTP 403 and a rendered page; unauthenticated users are redirected to login.

Verified: **every controller action calls it**. The student controller does so through a shared private helper:

```
private function bootStudentContext(): array
{
    AuthMiddleware::requireRole('student');
    // ... resolve user → student profile → current class → school settings
    return ['student' => $student, 'studentId' => $studentId,
           'classId' => $classId, 'school' => $school];
}
```

Each of the four student pages begins with `$context = $this->bootStudentContext();`. This is the right pattern: authorization cannot be omitted from a new student page without also omitting the data the page needs, so the two failure modes collapse into one that is immediately visible.

Ownership checks beyond role

Role alone is insufficient — one teacher must not mark another's class, and one parent must not read another family's child. Both are handled:

```
// TeacherController – mark attendance / enter scores
$assignment = $this->assignmentModel->find($assignmentId);
if (!$assignment || (int) $assignment['teacher_id'] !== $teacher['id']) {
    http_response_code(403);
    echo "Access denied. This is not your assignment.";
    exit;
}

// ParentController – child detail
if (!$this->parentModel->isLinkedToChild($parent['id'], $studentId)) {
    http_response_code(403);
    echo "Access denied. This is not your child.";
    exit;
}
```

These are Insecure Direct Object Reference defences, and they are correctly placed *before* any data is loaded. This is the class of check most projects of this size omit entirely.

7.3 Forced password change

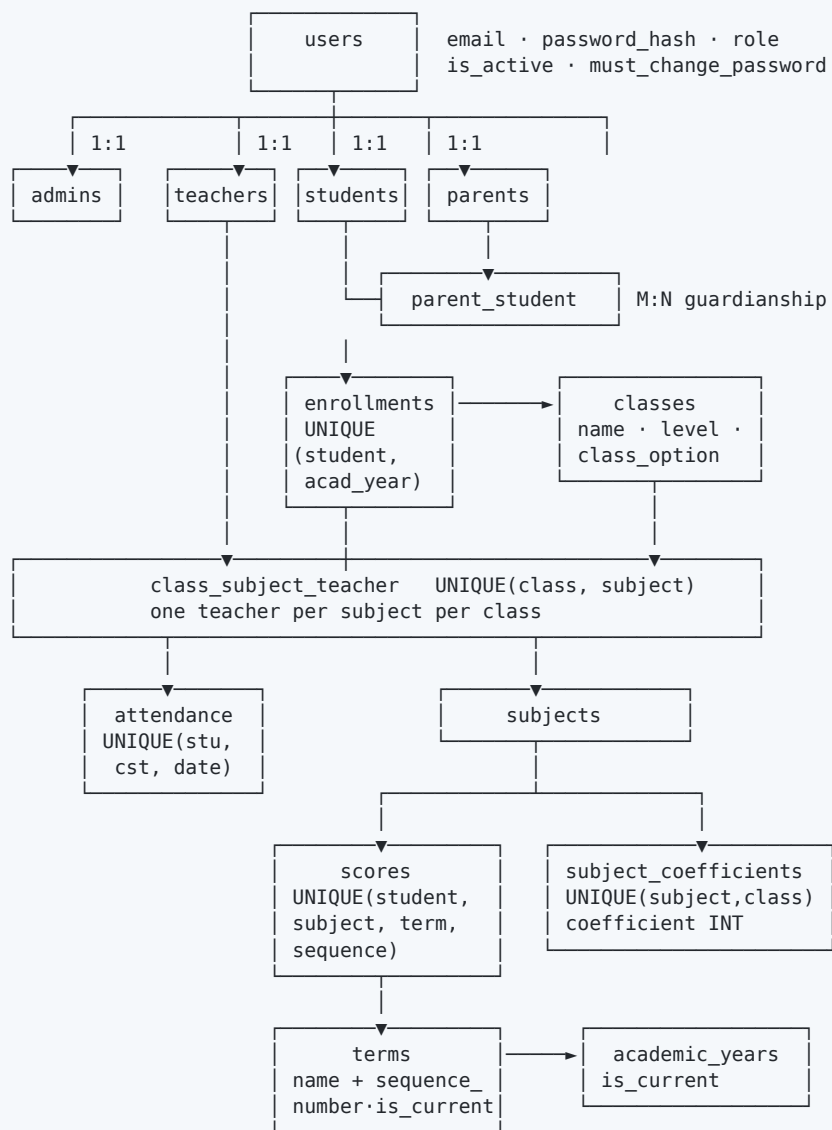
Accounts are created with `must_change_password = TRUE`. The middleware redirects such users to the change-password page on every request — explicitly exempting `change_password_form` and `change_password` themselves, which is what prevents an infinite redirect loop. The exemption is the subtle part; do not remove it.

7.4 CSRF, rate limiting and session hardening

Control	Implementation
CSRF token	32 random bytes from <code>random_bytes()</code> , hex-encoded, stored per session. Emitted by <code>Security::csrfField()</code> , compared with <code>hash_equals()</code> — constant-time, so the comparison cannot be attacked by timing.
CSRF failure handling	HTTP 403, then redirect back to the referring page with an error message — but only when the referer contains <code>BASE_URL</code> , so the response cannot be used as an open redirect.
Password policy	<code>validatePasswordStrength()</code> : minimum 8 characters, at least one uppercase, one lowercase, one digit. Verified applied consistently at teacher, student and parent creation <i>and</i> at password change.
Password storage	<code>password_hash(\$password, PASSWORD_DEFAULT)</code> — bcrypt today, automatically upgraded by PHP in future versions.
Rate limiting	5 attempts, 5-minute lockout, counted in the session. See §13.4 for the limitation this carries.
Session cookie	<code>httponly</code> (no JavaScript access), <code>samesite=Strict</code> (not sent cross-site), <code>use_only_cookies</code> (no session ID in URLs), <code>use_strict_mode</code> (rejects attacker-supplied IDs), and <code>secure</code> when HTTPS is detected.

8. Database Design

8.1 Entity-relationship overview



Standalone: grade_scale · announcements · school_settings

8.2 Table reference

Table	Key constraints	Purpose
users	UNIQUE(email) ; role ENUM of 4	Single auth table for all roles. Profile detail lives in the four role tables.
admins , teachers , students , parents	UNIQUE(user_id) , FK ON DELETE CASCADE	Role profiles. Deleting a user removes the profile automatically.
parent_student	Composite PK (parent_id, student_id)	Many-to-many guardianship — one parent, several children; one child, several guardians.
classes	—	name ("Form 4 Science A"), level ("Form 4"), class_option (Science/Arts/Commercial, NULL in lower forms).
subjects	UNIQUE(code)	Subject catalogue.
class_subject_teacher	UNIQUE(class_id, subject_id)	The permission table. A teacher's rows here define exactly what they may mark. Enforces one teacher per subject per class.
academic_years	UNIQUE(name) , is_current	Exactly one row should be current; enforced in application code, not by the database.
terms	UNIQUE(academic_year_id, name, sequence_number)	One row per <i>term-sequence</i> — see §8.3. Also carries is_current .
enrollments	UNIQUE(student_id, academic_year_id)	Which class a student sits in for a given year. Supports promotion across years while preserving history.
attendance	UNIQUE(student_id, class_subject_teacher_id, date)	Per-subject-period register. The unique key makes re-marking idempotent.
scores	UNIQUE(student_id, subject_id, term_id, sequence)	Raw marks. Averages are never stored — always derived.
grade_scale	—	Admin-editable letter bands, seeded A-F on the Cameroon scale. Grading policy can change without touching historical scores.
subject_coefficients	UNIQUE(subject_id, class_id)	Per-class weighting used by the overall average.
announcements	class_id NULL = school-wide	Notices scoped to a class or the whole school.
school_settings	Single row	Branding, colours and pass_mark . See §14.2.

Derived, never stored. No average, grade, position or total is persisted anywhere. Every such figure is computed from raw `scores` rows on each request. That is the correct trade-off at school scale: it costs some CPU, but it makes it impossible for a stored average to silently disagree with the marks behind it after a correction. Keep it until measurements prove otherwise — then cache, do not denormalise.

8.3 The terms/sequence modelling decision

This is the single most important thing to understand before touching the grading code.

A row in `terms` is **not a term** — it is a *term-sequence*. "Term 1" with two sequences is two rows:

id	academic_year_id	name	sequence_number	is_current
1	1	Term 1	1	0
2	1	Term 1	2	1
3	1	Term 2	1	0

A term is therefore identified by the pair `(academic_year_id, name)`, not by a single id. This is why `Score::subjectAveragesForStudentTerm()` contains a self-join that looks unnecessary at first reading:

```
INNER JOIN terms selected_term ON selected_term.id = :term_id
WHERE sc.student_id = :student_id
  AND sc.term_id IN (
    SELECT t.id FROM terms t
    WHERE t.name = selected_term.name
      AND t.academic_year_id = selected_term.academic_year_id
  )
```

It takes the given term-sequence row, recovers its name and year, and then averages across *all* sequences of that term. Without it, a "term average" would only ever reflect one sequence. Any new query that aggregates by term must reproduce this pattern — and §14.4 documents the one place that does not.

Redundancy to be aware of. `scores.sequence` and `terms.sequence_number` both record the sequence. Nothing in the schema forces them to agree, so a row can claim `sequence = 1` while its `term_id` points at the sequence-2 term row. The application never writes such a row today, but a future bulk import could. Longer term, the cleaner model is a `terms` table holding three real terms and letting `scores.sequence` be the only sequence marker. That is a migration, not a quick fix — plan it, do not improvise it.

8.4 Migrations

File	Effect
<code>migration_001_sync_schema.sql</code>	Adds <code>users.must_change_password</code> , <code>terms.is_current</code> , <code>attendance.class_subject_teacher_id</code> , and creates <code>school_settings</code> and <code>subject_coefficients</code> . Idempotent via <code>IF NOT EXISTS</code> .
<code>migration_002_pass_mark.sql</code>	Adds <code>school_settings.pass_mark DECIMAL(5,2) DEFAULT 10.00</code> . Its own comment states "The Result model reads this to decide PASS/FAIL" — it does not. See §14.2.
<code>migration_003_terms_unique_constraint.sql</code>	Adds <code>UNIQUE(academic_year_id, name, sequence_number)</code> on <code>terms</code> , preventing duplicate term rows from splitting a student's scores across two ids and silently corrupting averages. Documents its precondition query and its rollback — this is exactly how a migration should be written.

Two gaps in the migration story. First, `migration_001` leaves the `attendance` foreign key and unique-index changes commented out with a note to "run these manually", so two databases that both ran every migration can still differ. Second, nothing records which migrations have been applied. A four-line `schema_migrations(version, applied_at)` table and a short runner script would remove both problems permanently.

9. The Academic Engine

This is the core of the product and deserves the closest reading. Four classes cooperate: `Score` (761 lines), `Result` (462), `SubjectCoefficient` (280) and `GradeScale` (22).

9.1 Score entry

`Score::recordBulk()` saves a whole class's marks for one subject, term and sequence:

```
INSERT INTO scores (student_id, subject_id, term_id, sequence, score, max_score)
VALUES (:student_id, :subject_id, :term_id, :sequence, :score, 20.00)
ON DUPLICATE KEY UPDATE score = VALUES(score), max_score = VALUES(max_score)
```

Four properties are worth naming:

- **Idempotent.** `ON DUPLICATE KEY UPDATE` combined with the unique key means re-submitting the form corrects marks rather than duplicating them.
- **Transactional.** The whole batch is wrapped in `beginTransaction()` / `commit()`, with rollback guarded by `inTransaction()`. One invalid mark rejects the entire batch — the correct choice, since a half-saved register is worse than a rejected one.
- **Validated.** Non-numeric values and marks outside 0–20 throw `InvalidArgumentException`; the identifiers are rejected if non-positive.
- **Blank means ungraded.** Empty strings are skipped, not stored as zero. This matters: a missing mark and a mark of zero are different facts, and conflating them would drag a student's average down.

9.2 Subject averages and term aggregation

`Score::subjectAveragesForStudentTerm($studentId, $termId)` returns one row per subject with `average_score`, `sequence_count` and `max_score`, averaging across every sequence of the term via the self-join described in §8.3. This is the method the report card renders.

9.3 Coefficient-weighted overall average

`Score::overallAverageForStudentTerm($studentId, $termId, $classId = 0)` implements the Cameroon formula:

```
$coefficients = $classId > 0
  ? (new SubjectCoefficient())->getForClass($classId) // [subject_id => coefficient]
  : [];

foreach ($subjects as $subject) {
    if ($subject['average_score'] === null) continue;

    $coefficient = $coefficients[(int) $subject['subject_id']] ?? 1;

    $weightedTotal    += (float) $subject['average_score'] * $coefficient;
    $totalCoefficients += $coefficient;
}

return $totalCoefficients === 0
    ? null
    : round($weightedTotal / $totalCoefficients, 2);
```

The `?? 1` default is the important detail: a subject with no configured coefficient falls back to weight 1, so the formula degrades gracefully to a plain mean rather than dropping the subject or dividing by zero. Ungraded subjects are skipped entirely, and an empty result returns `null` — not `0`, which would be indistinguishable from a genuine zero average.

This is correct, and it is the feature that makes the system usable in a Cameroonian school. Both project notes still list weighted averages as an unimplemented gap; that is stale (§15).

9.4 Grade scale

Letters and remarks are never hard-coded. `Result::getGrade()` and `GradeScale::forScore()` both query the `grade_scale` table:

```
SELECT id, min_score, max_score, letter, remark
FROM grade_scale
WHERE :score >= min_score AND :score <= max_score
ORDER BY min_score DESC
LIMIT 1
```

Seeded bands: A 16–20 Excellent, B 14–15.99 Very Good, C 12–13.99 Good, D 10–11.99 Fair, E 8–9.99 Weak, F 0–7.99 Fail. An administrator can change the policy without touching a single stored score.

A latent portability trap. This query binds the named placeholder `:score` twice. PDO's *emulated* prepares — the MySQL default, and what this project runs with — expand it fine. Native prepares do not: reusing a named placeholder raises `HY093 Invalid parameter number`. So the day someone hardens the connection with `PDO::ATTR_EMULATE_PREPARES => false` (normally good advice), every grade lookup in the system breaks. Fix it pre-emptively by binding two distinct placeholders, `:score_min` and `:score_max`, with the same value.

9.5 Class rankings

`Result::getClassResults($classId, $academicYearId, $termId)` builds the ranking:

1. Load every student enrolled in the class for that academic year.
2. Compute each student's overall result — delegating the average to `Score::overallAverageForStudentTerm()`.
3. `usort()` by average descending, with students who have no results forced to the bottom and ties broken by name.
4. Assign competition-style positions: equal averages share a position and the next distinct average skips accordingly (1st, 2nd, 2nd, 4th). Students without results get `position = null` rather than a misleading last place.

One definition of "average", by construction. The ranking does not re-implement averaging — it calls the same `Score` method the report card calls. This is deliberate and the code comments say so. It means a student's displayed average and the average their rank was computed from can never drift apart. If you ever need to change the averaging rule, change it in `Score` only; do not add a second implementation.

Performance: this is an N+1 pattern. Ranking a class runs roughly three queries per student — subject results, subject averages, and the coefficient lookup, the last of which fetches the same per-class coefficient map once per student. A 60-student class costs around 180 queries, and the student report card triggers a full class ranking just to find one position. It is tolerable at a single school's scale, but it is the first thing that will hurt. Two cheap fixes: hoist the coefficient lookup out of the loop (it is identical for every student in the class), and compute the whole class's subject averages in one grouped query instead of one query per student.

10. Feature Walkthrough by Role

10.1 Administrator

Capability	Detail
Dashboard	Counts of students, teachers, classes and related statistics.
Academic year lifecycle	Create a year, activate it, and set the current term/sequence. <code>Term::setCurrent()</code> runs in a transaction: it verifies the term belongs to the current year, clears <code>is_current</code> across that year, sets the new one, and throws if no row was updated. That is the correct way to maintain a single-current-row invariant.
Create teacher	User account + teacher profile, password strength enforced.
Create student	User + student profile + enrolment, wrapped in an explicit transaction (<code>AdminController</code> lines 416–486) so a failure cannot leave an orphaned account with no enrolment.
Create parent	User + parent profile + child links.
Assign teachers	Creates the <code>class_subject_teacher</code> row that grants marking rights.
Class coefficients	Per-class subject weighting, saved in bulk within a transaction.
Announcements	Post school-wide or class-scoped notices; view all.
Listings	Read-only views of teachers, classes and class rosters.

The biggest functional gap in the product. Every entity can be created; none can be edited, deactivated or deleted. A mistyped student name is permanent through the UI, a teacher who leaves cannot be deactivated, and a wrong teacher assignment cannot be corrected. The `users.is_active` column already exists and login already honours it — a small user-management screen would unlock most of this value immediately. This is the highest-value feature work available.

10.2 Teacher

The dashboard lists the teacher's `class_subject_teacher` assignments plus announcements. From each assignment they can mark attendance for today or enter scores for a chosen term and sequence. Both actions re-verify assignment ownership server-side before doing anything (§7.2). The score form only loads the class roster once a term/sequence has been selected, which avoids showing a marking grid that cannot yet be saved.

10.3 Student

The originally monolithic dashboard has been split into four focused pages — Information, Report Card, Attendance, Announcements — all sharing `bootStudentContext()`. The legacy `student_dashboard` route now issues an HTTP 301 to the information page so old bookmarks keep working. Using 301 rather than 302 here is the right call for a permanent split.

The report card shows per-subject averages with letter and remark, the coefficient-weighted overall average, the overall grade, and class position out of the number of students who actually have results — a more honest denominator than total enrolment.

10.4 Parent

The dashboard lists linked children. Selecting one opens a detail page with that child's attendance, scores, report card, overall average, class position and class-scoped announcements — gated by the `isLinkedToChild()` check described in §7.2.

11. Front-End Architecture

Bootstrap 5.3.3 arrives by CDN and is supplemented by five local stylesheets, each with a clear remit:

File	Lines	Responsibility
theme.css	463	Design tokens — the navy #123B63 and gold #D69E2E palette, typography, spacing.
components.css	1,323	Buttons, cards, badges, tables, forms.
dashboard.css	931	Sidebar, header, stat cards, dashboard grid.
auth.css	1,930	Login and password-change screens.
responsive.css	293	Breakpoints and mobile behaviour.

The brand colours are also stored in `school_settings.primary_color / secondary_color`, which suggests per-school theming was intended. The CSS does not yet read them — exposing them as CSS custom properties on `:root` from the settings row would complete that feature cheaply.

JavaScript — `public/js/app.js` (944 lines)

One file, no framework, no build step, organised as discrete `initialize*()` functions invoked on `DOMContentLoaded` :

- `initializeAlerts()` — dismissible flash messages.
- `initializeFormSubmitProtection()` — disables the submit button on submit, preventing double-posted score batches.
- `initializeDirtyPostFormWarning()` — warns before navigating away from an edited form. Genuinely valuable when a teacher has typed forty marks.
- `initializeDashboardSidebar()` — responsive sidebar with overlay and keyboard handling.
- `initializeStatusBadges()` / `initializeGradeBadges()` — colour-codes attendance and grade cells by reading the table's column headers.
- `initializePrintButton()` — report-card printing.

The code is defensive throughout — it checks that elements exist before binding — so a page missing a sidebar does not throw. That is the right style for progressively enhanced server-rendered pages.

Notes on the front-end

- `auth.css` is 1,930 lines for two screens. That is disproportionate and almost certainly contains dead rules; a pass with browser coverage tooling would likely halve it.
- `views/components/sidebar.php` is 1,195 lines, the largest view in the project. It renders four role-specific navigations in one file. Splitting it into four partials selected by role would make each one readable.
- Output escaping in views is disciplined — 167 `htmlspecialchars(..., ENT_QUOTES, 'UTF-8')` calls, with integers cast `(int) ($term['id'] ?? 0)` before being echoed unescaped. The few unescaped echoes reviewed were all ternaries over literal strings or integer counters, which are safe.

12. Installation & Setup

A fresh clone will not run. `config/database.php` begins with `require_once __DIR__ . '/env.php';`, but `config/env.php` is listed in `.gitignore` and is not in the repository. Every request fails before reaching the router. Step 4 below is the workaround; §14.1 is the real fix.

1. **Start XAMPP** — Apache and MySQL.
2. **Clone into the web root**, e.g. `C:\xampp\htdocs\school-system`. The folder name must match the path segment in `BASE_URL`.
3. **Create the database and schema:**

```
CREATE DATABASE `school-system`  
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  
  
-- then, in that database:  
SOURCE database/schema.sql;  
SOURCE database/migration_001_sync_schema.sql;  
SOURCE database/migration_002_pass_mark.sql;  
SOURCE database/migration_003_terms_unique_constraint.sql;
```

4. **Recreate `config/env.php`** — it must define an `env($key, $default)` function that reads `.env`. See §14.1 for why this file belongs in version control and `.env` does not.
5. **Copy `.env.example` to `.env`** and set the real values:

```
DB_HOST=localhost  
DB_NAME=school-system  
DB_USERNAME=root  
DB_PASSWORD=  
BASE_URL=/school-system/public  
SCHOOL_EMAIL_DOMAIN=@school.com
```

6. **Seed the first administrator.** There is no installer and no registration page, so the first account must be created by hand. Generate a hash with `php -r "echo password_hash('YourStrongPass1', PASSWORD_DEFAULT);"`, then:

```
INSERT INTO users (email, password_hash, role, is_active, must_change_password)  
VALUES ('admin@school.com', '<paste the hash>', 'admin', 1, 1);  
  
INSERT INTO admins (user_id, full_name)  
VALUES (LAST_INSERT_ID(), 'System Administrator');
```

7. **Open** `http://localhost/school-system/public/index.php?action=login`.

Smoke test after setup. Log in as admin → forced password change appears → create an academic year and activate it → set a current term/sequence → create a class, a subject, a teacher and a student → assign the teacher to that class+subject → set coefficients → log in as the teacher → enter marks for both sequences of a term → log in as the student → confirm the report card shows the subject average, the weighted overall average, a grade and a class position. That path exercises nearly every component of the system; if it passes, the installation is sound.

13. Security Review

Findings are ordered by severity, each with root cause before remedy.

13.1 **CRITICAL** Source and secrets are exposed over HTTP

Root cause: `.htaccess` at the project root is **0 bytes**, and the documented `BASE_URL` is `/school-system/public` — meaning Apache serves the entire project directory, not just `public/`. Nothing restricts access to the parent folders.

Impact: with the project at `htdocs/school-system`, these are all fetchable by anyone who can reach the server:

```
http://host/school-system/.env           → database credentials
http://host/school-system/config/database.php → served as PHP, but...
http://host/school-system/database/schema.sql → full schema, downloaded as text
http://host/school-system/src/Models/Score.php → source if PHP handling is ever misconfigured
```

The `.sql` files and `.env` are not PHP, so Apache returns them as plain text immediately. Database credentials leak on the first request.

Fix — do both:

1. **Point the virtual host's** `DocumentRoot` **at** `public/`. This is the real fix. Nothing outside `public/` is then addressable at all, and `BASE_URL` becomes `/`.
2. **Add defence in depth** in the root `.htaccess`, for shared hosting where you cannot set the document root:

```
# Block everything at the project root except the public/ entry point
<FilesMatch "\.(env|sql|md|json|lock|ini|log)$">
    Require all denied
</FilesMatch>

RedirectMatch 403 ^/school-system/(config|src|database|views)/.*$
```

Treat any credentials that have been live on an exposed host as compromised and rotate them.

13.2 **STRONG** Injection, CSRF and XSS defences

These are done properly and are recorded here so a future reviewer does not "fix" them into something worse:

- **SQL injection** — every query in all 16 models uses PDO prepared statements with named parameters. No string interpolation into SQL was found anywhere in the codebase.
- **CSRF** — validated once, globally, for every POST before routing (\$5.2). Tokens are 32 random bytes compared with `hash_equals()`.
- **XSS** — 167 escaped outputs with `ENT_QUOTES` and explicit UTF-8; integers cast before echo.
- **Session fixation** — `session_regenerate_id(true)` on successful login.
- **IDOR** — ownership verified for teacher assignments and parent-child links.
- **User enumeration** — identical error message whether or not the email exists.

13.3 MEDIUM Database errors and internal messages reach the browser

Root cause: `config/database.php` ends its catch block with `die("Database connection failed: " . $e->getMessage())`. Several controllers likewise `echo` a message and `exit` on missing profiles.

Impact: a connection failure prints the host, database name and driver error to the visitor.

Fix: log with `error_log()`, render a neutral page, and in production set `display_errors = 0` with `log_errors = 1`. Replace the bare `echo` + `exit` paths with a small shared error view so failures look like the rest of the application.

13.4 MEDIUM Rate limiting is per-session, not per-account

Root cause: the attempt counter lives in `$_SESSION`.

Impact: an attacker who discards the session cookie between attempts gets an unlimited allowance. The control stops an honest user fat-fingering a password; it does not stop a scripted attack.

Fix: record attempts in a `login_attempts(email, ip_address, attempted_at)` table and throttle on the email/IP pair. The existing session check can stay as a cheap first line.

13.5 LOW CSRF token never rotates

The token is generated once per session and reused until logout. Rotating it after each successful state-changing POST narrows the window in which a leaked token is useful. Low priority given the `samesite=Strict` cookie already in place.

13.6 LOW No security headers

No `Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options` or `Referrer-Policy` is sent. Adding them in `.htaccess` costs four lines and mitigates clickjacking, MIME sniffing and referrer leakage. Note that a strict CSP will require removing the inline `style="..."` attributes and `onclick` handlers currently present in several views.

Security scorecard

Control	Status	Reference
Password hashing	Strong	password_hash / password_verify
SQL injection defence	Strong	PDO prepares everywhere
CSRF protection	Strong	Global gate, §5.2
XSS defence	Strong	167 escaped outputs
Session hardening	Strong	§7.4
Role authorization	Strong	Middleware on every action
Object-level authorization	Strong	§7.2
Web-root isolation	Critical	§13.1
Error disclosure	Weak	§13.3
Brute-force protection	Partial	§13.4
Security headers	Absent	§13.6
Audit logging	Absent	No record of who changed a mark

14. Verified Defects & Root-Cause Analysis

Every defect below was confirmed against the source at commit `e9f23ca`. Claims from the project's own notes that did *not* survive verification are listed separately in §15.

14.1 **CRITICAL** A fresh clone cannot boot — `config/env.php` is git-ignored

Where: `.gitignore` lines 3–6; `config/database.php` line 18.

```
# .gitignore
.env
config/env.php      ← this is the problem
```

Root cause: two different kinds of file were treated as one. `.env` holds *secrets* and must never be committed. `config/env.php` is the *parser* that reads `.env` — it is ordinary application code containing no secrets, and `config/database.php` requires it unconditionally on line 18. Ignoring it means the repository is missing a file it cannot run without.

Symptom: Fatal error: Failed opening required `'../config/env.php'` on every request after a clean clone. Verified: the file is absent from the repository, and `.env.example` is the only env-related file tracked.

Fix: remove `config/env.php` from `.gitignore` and commit it. Keep `.env` ignored — that line is correct and should stay. Also note `.env` is listed twice in the file; harmless, but tidy it. If `env.php` has been lost locally, it needs to be re-authored:

```
<?php
declare(strict_types=1);

function env(string $key, ?string $default = null): ?string
{
    static $vars = null;

    if ($vars === null) {
        $vars = [];
        $path = __DIR__ . '/../.env';

        if (is_readable($path)) {
            foreach (file($path, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES) as $line) {
                $line = trim($line);
                if ($line === '' || $line[0] === '#' || !str_contains($line, '=')) {
                    continue;
                }
                [$name, $value] = explode('=', $line, 2);
                $vars[trim($name)] = trim($value, " \t\n\r\0\x0B\"'");
            }
        }
    }

    return $vars[$key] ?? $default;
}
```

This is a reconstruction consistent with how `config/database.php` calls `env()`, not a copy of the original file — the original is not in the repository to compare against.

14.2 **HIGH** Pass/fail uses a hard-coded 8.00 and ignores the configured pass mark

Where: `src/Models/Result.php`, `determinePassFail()`.

```
public function determinePassFail(float $average): string
{
    return $average >= 8.00 ? 'PASS' : 'FAIL';
}
```

Root cause: the method predates `migration_002_pass_mark.sql`, which added `school_settings.pass_mark DECIMAL(5,2) NOT NULL DEFAULT 10.00`. That migration's own header states: *"The Result model reads this to decide PASS/FAIL."* It does not — the constant was never replaced. `SchoolSettings::get()` already selects the column and even falls back to `'10.00'`, so the value is available and simply unused.

Impact: this is a correctness bug with real consequences, not a style issue. The Cameroon pass mark is 10/20, and the seeded grade scale marks 8–9.99 as "E — Weak". Every student averaging between 8.00 and 9.99 is currently reported as **PASS** when the school's own configured threshold says they have failed. It also silently propagates into `getClassSummary()`, inflating the reported pass rate.

Fix: read the configured value, defaulting to 10.00:

```
public function determinePassFail(float $average): string
{
    $passMark = (float) (School::settings()['pass_mark'] ?? 10.00);

    return $average >= $passMark ? 'PASS' : 'FAIL';
}
```

Then delete the now-inaccurate docblock above the method, which still describes the 8.00 rule as "the current school rule".

14.3 **HIGH** A view include breaks on Linux due to file casing

Where: `src/Controllers/TeacherController.php` line 77.

```
require __DIR__ . '/../views/teacher/mark_attendance.php'; // lowercase m
// file on disk:  views/teacher/Mark_attendance.php        // uppercase M
```

Root cause: Windows and macOS default to case-insensitive filesystems, so the mismatch is invisible during XAMPP development. Linux — where this will be deployed — is case-sensitive.

Impact: the teacher attendance page fatals on any Linux host. Verified by resolving every `require / include` path in the project against the actual filesystem; this was the only genuine casing mismatch found.

Fix: rename the file, preserving history and forcing Git to record the case change:

```
git mv views/teacher/Mark_attendance.php views/teacher/mark_attendance.tmp
git mv views/teacher/mark_attendance.tmp views/teacher/mark_attendance.php
```

The two-step rename is necessary because Git on a case-insensitive filesystem ignores a direct case-only rename.

Prevent the whole class of bug with a CI check that resolves every include path against the filesystem on Linux. Three lines in a workflow will catch this permanently — see §16, Phase 1.

14.4 **MEDIUM** Two different "subject results" definitions coexist

Where: `Score::subjectAveragesForStudentTerm()` versus `Result::getStudentSubjectResults()`.

Root cause: the `Score` version aggregates across every sequence sharing the term's name and academic year (§8.3). The `Result` version filters on `sc.term_id = :term_id` exactly — a single sequence. The two were written at different times and the second never adopted the aggregation rule.

Impact: the overall average is computed from the aggregated set, but `subject_count` inside `getStudentOverallResult()` is counted from the non-aggregated one. A student with marks in both sequences can therefore be shown an overall average derived from, say, eight subjects alongside a subject count of five. The overall figure itself is correct; the count beside it is not, and the per-subject grades returned by `getStudentClassPosition()` reflect one sequence rather than the term.

Fix: delete `Result::getStudentSubjectResults()` and have `Result` call `Score::subjectAveragesForStudentTerm()`, exactly as it already does for the overall average. One definition of a subject average should exist in the codebase, and `Score` is the right home for it.

14.5 **MEDIUM** `User::updatePassword()` cannot report failure

Where: `src/Models/User.php` and `AuthController::changePassword()`.

```
// Model – declared void, returns nothing
public function updatePassword(int $userId, string $newPassword): void

// Controller – checks a return value that can never be false
$updated = $this->userModel->updatePassword(...);
if ($updated === false) { /* unreachable */ }
```

Root cause: the model's contract and the controller's expectation disagree. A `void` function evaluates to `null`, and `null === false` is `false`, so the guard never fires.

Impact: limited in practice, because PDO is configured with `ERRMODE_EXCEPTION`, so a genuine database failure throws rather than returning quietly. But the controller contains dead error-handling that gives a false impression of safety, and if a caller ever uses a driver without exception mode, a failed password change reports success.

Fix: change the signature to `: bool` and `return $stmt->execute([...]);`. The controller's existing check then works as written.

14.6 **MEDIUM** "Latest term" is chosen by highest id, not by date

Where: `Score::latestTermIdForStudent()` — `ORDER BY term_id DESC LIMIT 1`.

Root cause: the query assumes a higher `terms.id` always means a chronologically later term-sequence. That holds only while terms are inserted in calendar order and never backfilled.

Impact: if a term is ever added out of order — correcting a missed year, importing history, re-seeding — students' report cards silently display the wrong term.

Fix: add `start_date / end_date` to `terms` and order by date; or, as a smaller step, join `academic_years` and order by `(ay.name DESC, t.name DESC, t.sequence_number DESC)`, which encodes the real intent rather than relying on insertion order.

14.7 **LOW** Grade lookup breaks if prepared-statement emulation is disabled

Covered in §9.4. Two distinct placeholders instead of one reused `:score`.

14.8 **LOW** Ranking is an N+1 query pattern

Covered in §9.5. Hoist the per-class coefficient lookup out of the per-student loop and batch the subject averages.

Fix priority

Order	Item	Why this order
1	Restrict the web root (§13.1)	Credentials are readable over HTTP. Nothing else matters while this is true.
2	Commit <code>config/env.php</code> (§14.1)	Nobody else can run the project until this is fixed.
3	Rename <code>Mark_attendance.php</code> (§14.3)	Blocks any Linux deployment; two commands to fix.
4	Read the configured pass mark (§14.2)	The system currently passes students the school considers failed.
5	Unify the subject-average definition (§14.4)	Removes the second source of truth before more code depends on it.
6	Fix <code>updatePassword()</code> return type (§14.5)	Small, and removes misleading dead error handling.
7	Admin edit / deactivate screens (§10.1)	Highest-value missing feature; the data model already supports it.
8	Migration tracking table (§8.4)	Prevents environments drifting apart as the schema grows.

15. Documentation Drift

CLAUDE.md and PROJECT_PROGRESS.md are useful and unusually well maintained for a solo project — but parts have fallen behind the code. A new developer trusting them would chase problems that no longer exist and miss ones that do. Verified status of every claim in their "Known Issues" and "Critical Findings" sections:

Documented claim	Verified	Evidence
"Model files have lowercase names but requires use PascalCase"	Fixed	All 16 model filenames and all require paths resolve correctly.
" Mark_attendance.php has mixed case"	Still broken	§14.3 — the only real casing mismatch left.
"Weighted averages not implemented"	Implemented	Score::overallAverageForStudentTerm() , §9.3.
"Result model exists but is not wired to any route"	Wired	Used by StudentController::reportCard() and ParentController::childDetail() .
"SubjectCoefficient is not used in any calculation"	In use	Called from the weighting loop, §9.3.
" Subject::getDashboardStatistics() is dead code"	Removed	Subject.php now contains only all() .
" User::updatePassword() returns void but caller checks false"	Still present	§14.5.
"Password policy inconsistent — creation allows 6 chars"	Consistent	validatePasswordStrength() applied at all three creation paths and at change.
"Layout fragmentation — only admin dashboard uses the shared layout"	Complete	All 23 page views use views/layouts/dashboard.php .
"Phase 1: Fixed Student::create() — added admission_no "	Not present	admission_no appears nowhere in the codebase or schema; the change was reverted or lost.

Recommendation. Rewrite the "Known Issues" section of CLAUDE.md from §14 of this document, and move the completed items in PROJECT_PROGRESS.md from "next" to "done". Stale documentation is worse than none: it costs a new developer a day proving that fixed bugs are fixed, and it hides the four that are real. Going forward, treat a "Known Issues" entry as something that is removed in the same commit that fixes it.

16. Improvement Roadmap

Four phases, ordered so each rests on the last. Phase 1 is the whole of the blocking work and is small.

Phase 1 — Unblock deployment (days)

- Move the document root to `public/`; add the defensive `.htaccess` (§13.1).
- Commit `config/env.php`; keep `.env` ignored (§14.1).
- Rename `Mark_attendance.php` (§14.3).
- Read `school_settings.pass_mark` in `determinePassFail()` (§14.2).
- Fix `updatePassword()` 's return type (§14.5); unify subject averages (§14.4).
- Replace `die()` on connection failure with logging plus a neutral error page (§13.3).
- Add a CI workflow — `php -l` across every file, plus an include-path resolution check on Linux, which would have caught §14.3 automatically:

```
name: CI
on: [push, pull_request]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with: { php-version: '8.2' }
      - name: Syntax check
        run: find . -name "*.php" -print0 | xargs -0 -n1 php -l
      - name: Verify include paths resolve (case-sensitive)
        run: |
          grep -rhoP "(require|include)(\_once)?\$+__DIR__\$*\.\$s*'\[^']+'" \
            --include="*.php" src public views | sort -u
```

Phase 2 — Complete the administrative surface (weeks)

- Edit and deactivate for students, teachers, parents and classes — the largest missing capability (§10.1).
- A user-management list: search, filter, activate/deactivate, reset password.
- CRUD for subjects and classes; a school-settings editor; a grade-scale editor.
- Search, filter and pagination on every listing. No list in the system is paginated today.
- An `audit_log` table recording who changed which mark and when. In a school, score changes are disputed — without this there is no answer.

Phase 3 — Results workflow and reporting (weeks)

- Printable Cameroon-format report cards (HTML first, then PDF) — coefficient, sequence 1, sequence 2, term average, class average, rank and remarks per subject.
- Result approval and locking, so marks cannot change after publication without an audited unlock.
- Class result sheets and ranking sheets for staff meetings.
- A migration-tracking table and runner (§8.4).

Phase 4 — Scale, analytics and Cameroon-specific features (months)

- Replace the 707-line router `switch` with a route table (§6).
- Resolve the N+1 ranking queries (§9.5); add indexes once real query plans are measured.

- Attendance summaries, class and subject analytics, student profiles.
- Fees with Mobile Money (MTN MoMo / Orange Money) reconciliation, and SMS or WhatsApp notification to parents. Both fit how Cameroonian schools and families actually operate, and both are far easier to design in than to retrofit.
- Bilingual English/French labels — mandated in much of the country and structural, not cosmetic.
- Introduce Composer with PSR-4 autoloading to replace ~100 manual `require_once` lines, and PHPUnit tests over the grading engine — which is the part where a silent regression does the most damage.

Appendix A — Route Table

All 30 registered actions. Unknown actions render a styled 404.

Action	Method	Role	Controller method
login	GET	public	AuthController::showLoginForm
login_submit	POST	public	AuthController::login
logout	GET	any	AuthController::logout
change_password_form	GET	any	AuthController::showChangePasswordForm
change_password	POST	any	AuthController::changePassword
admin_dashboard	GET	admin	AdminController::dashboard
academic_years	GET	admin	AcademicYearController::index
create_academic_year	POST	admin	AcademicYearController::create
activate_academic_year	POST	admin	AcademicYearController::activate
set_current_term	POST	admin	AcademicYearController::setCurrentTerm
create_teacher_form	GET	admin	AdminController::showCreateTeacherForm
create_teacher	POST	admin	AdminController::createTeacher
create_student_form	GET	admin	AdminController::showCreateStudentForm
create_student	POST	admin	AdminController::createStudent
create_parent_form	GET	admin	AdminController::showCreateParentForm
create_parent	POST	admin	AdminController::createParent
assign_teacher_form	GET	admin	AdminController::showAssignTeacherForm
assign_teacher	POST	admin	AdminController::assignTeacher
class_coefficients_form	GET	admin	AdminController::showClassCoefficientsForm
save_class_coefficients	POST	admin	AdminController::saveClassCoefficients
view_teachers	GET	admin	AdminController::viewTeacherAssignments
view_classes	GET	admin	AdminController::viewClasses
view_class_list	GET	admin	AdminController::viewClassList
post_announcement_form	GET	admin	AdminController::showPostAnnouncementForm
post_announcement	POST	admin	AdminController::postAnnouncement
view_announcements	GET	admin	AdminController::viewAnnouncements
teacher_dashboard	GET	teacher	TeacherController::dashboard
mark_attendance_form	GET	teacher	TeacherController::showMarkAttendanceForm
mark_attendance	POST	teacher	TeacherController::markAttendance
enter_scores_form	GET	teacher	TeacherController::showEnterScoresForm
enter_scores	POST	teacher	TeacherController::enterScores
student_information	GET	student	StudentController::information
student_report_card	GET	student	StudentController::reportCard
student_attendance	GET	student	StudentController::attendance

student_announcements	GET	student	StudentController::announcements
student_dashboard	GET	student	301 → student_information
parent_dashboard	GET	parent	ParentController::dashboard
parent_child_detail	GET	parent	ParentController::childDetail

Appendix B — Model API Reference

The 16 models and their public methods. All SQL in the application lives here.

Model	Public methods
User	<code>create(email, password, role): int</code> · <code>findByEmail(): array false</code> · <code>verifyPassword(): bool</code> · <code>emailExists(): bool</code> · <code>updatePassword(): void</code> §14.5
Student	<code>create</code> · <code>enroll</code> · <code>all</code> · <code>allEnrolledInCurrentYear</code> · <code>allByClass</code> · <code>allByClassDetailed</code> · <code>findByUserId</code> · <code>findByUserIdWithEmail</code> · <code>find</code> · <code>getCurrentClassId</code> · <code>getCurrentEnrollment</code> · <code>getCurrentClass</code> · <code>getDashboardStatistics</code>
Teacher	<code>create</code> · <code>all</code> · <code>findByUserId</code> · <code>getTotalCount</code>
ParentModel	<code>create</code> · <code>linkChild</code> · <code>all</code> · <code>findByUserId</code> · <code>getChildren</code> · <code>isLinkedToChild</code> IDOR guard
Score	<code>recordBulk</code> transactional · <code>getForSubjectTermSequence</code> · <code>getForSubjectTerm</code> · <code>forStudent</code> · <code>forStudentTerm</code> · <code>forStudentTermSequence</code> · <code>subjectAveragesForStudentTerm</code> · <code>overallAverageForStudentTerm</code> weighted · <code>latestTermIdForStudent</code> §14.6
Result	<code>getGrade</code> §9.4 · <code>getClassStudents</code> · <code>getStudentSubjectResults</code> §14.4 · <code>getStudentOverallResult</code> · <code>determinePassFail</code> §14.2 · <code>getClassResults</code> · <code>getStudentClassPosition</code> · <code>getClassSummary</code>
Term	<code>all</code> · <code>getCurrentTerm</code> · <code>find</code> · <code>findCurrent</code> · <code>getSequences</code> · <code>forAcademicYear</code> · <code>getCurrentAcademicYear</code> · <code>setCurrent</code> transactional
AcademicYear	<code>getCurrent</code> · <code>find</code> · <code>all</code> · <code>create</code> · <code>setCurrent</code> transactional
SubjectCoefficient	<code>getForSubjectClass</code> · <code>getForClass</code> · <code>getSubjectsForClass</code> · <code>save</code> · <code>saveBulk</code> transactional · <code>delete</code>
ClassSubjectTeacher	<code>assign</code> · <code>existsForClassSubject</code> · <code>all</code> · <code>forTeacher</code> · <code>subjectsForClass</code> · <code>find</code>
Attendance	<code>markBulk</code> · <code>getForAssignmentDate</code> · <code>forStudent</code>
Announcement	<code>create</code> · <code>forDashboard(?classId)</code> · <code>all</code>
GradeScale	<code>forScore(float): array false</code>
SchoolSettings	<code>get(): array</code> — includes <code>pass_mark</code> , with a <code>'10.00'</code> fallback
ClassModel	<code>all</code> · <code>getTotalCount</code> · <code>find</code>
Subject	<code>all</code>

Appendix C — Glossary

Term	Meaning in this project
Sequence	One of two assessment periods within a term. Six per academic year.
Term-sequence row	A row in <code>terms</code> . A term is the <i>pair</i> <code>(academic_year_id, name)</code> , not one row — §8.3.
Coefficient	Integer weight for a subject within a class, used by the overall average — §9.3.
GCE	General Certificate of Education — the Cameroonian Anglophone examination, O/L at Form 5 and A/L at Upper Sixth.
MINESEC	Cameroon's Ministry of Secondary Education, which standardises subject coefficients.
Front controller	A single entry point (<code>public/index.php</code>) through which every request passes before routing.
Middleware	A check that runs between routing and the controller body — here, <code>AuthMiddleware::requireRole()</code> .
CSRF	Cross-Site Request Forgery — making a logged-in user's browser issue a state-changing request. Blocked globally here.
IDOR	Insecure Direct Object Reference — reaching another user's record by changing an id in the URL. Guarded for teacher assignments and parent-child links.
N+1 queries	Running one query per item in a loop instead of one query for the whole set — §9.5.
Post/Redirect/Get	Redirecting after a successful POST so a refresh cannot resubmit it. Applied after every write.
Emulated prepares	PDO building the final SQL string in PHP rather than sending a parameterised statement to MySQL. On by default; §9.4 explains why that currently matters.

Every claim in this document was verified against the source at commit `e9f23ca`: include paths were resolved against the filesystem, all 74 PHP files were syntax-checked under PHP 8.4, and each defect in §14 was traced to a specific file and line. Where the project's own notes disagree with the code, §15 records which one is correct. Where a file or feature does not exist, this document says so rather than describing an assumed implementation.